

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS

TypeScript

JavaScript com Tipagem Estática e Segurança de Código

| CONCEITO E COMPILAÇÃO

Desenvolvido pela Microsoft, o TypeScript atua como uma ferramenta de desenvolvimento. Ele não é executado diretamente pelo navegador; ao invés disso, é **compilado** (transpilado) para JavaScript puro.

```
// Definição do contrato (tipo) antes da execução  
function exibirMensagem(texto: string): void {  
    console.log(texto);  
}
```

```
// O compilador verifica se o argumento é realmente uma string  
exibirMensagem("Olá TypeScript");
```


| PREVENÇÃO DE BUGS EM TEMPO DE ESCRITA

Em projetos de grande escala, bugs de tipo são comuns e caros. O TS detecta inconsistências de lógica e tipos durante o desenvolvimento, eliminando erros silenciosos de runtime.

// No JavaScript puro, isso geraria um bug silencioso (ex: 100100100)

// No TypeScript, o compilador barra o código imediatamente

```
function calcularPreco(valor: number, qtd: number): number {  
    return valor * qtd;  
}
```

calcularPreco("100", 5); // ERRO: String não pode ser tratada como Number

COMPARATIVO TÉCNICO

Característica	JavaScript	TypeScript
Tipagem	Dinâmica (muda em runtime)	Estática (verificada em build-time)
Detecção de Erros	Ao executar (Runtime)	Ao escrever (Compile-time)
Autocomplete	Limitado no editor	Completo e Inteligente (IntelliSense)
Escalabilidade	Complexa em grandes times	Alta manutenibilidade e previsibilidade

| TIPOS BÁSICOS

O TypeScript adiciona anotações claras, mas também possui **inferência de tipo**: se você inicializa uma variável, o TS "deduz" o tipo dela automaticamente.

```
let nome: string = "Ana";  
let ano = 2024; // Inferido como number  
  
// Uso do unknown para segurança em APIs  
let dados: unknown = obterDadosExternos();  
if (typeof dados === "string") {  
    console.log(dados.length); // Seguro: O TS agora sabe que é string  
}
```


| COLEÇÕES E CONSTANTES

As estruturas de dados ganham restrições de tipo, garantindo que listas e conjuntos de estados (Enums) sejam consistentes em toda a aplicação.

// Arrays tipados

```
const frutas: string[] = ["maçã", "banana"];
```

// Tuplas: Array com tamanho e tipos fixos

```
let posicao: [number, number] = [40.7, -74.0];
```

// Enum: Constantes nomeadas (mais legível em logs)

```
enum Status { Pendente = "p", Concluido = "c" }
```

```
let st: Status = Status.Concluido;
```


| MODELAGEM DE DADOS

Interfaces definem a estrutura de objetos e são extensíveis. **Type Aliases** são versáteis para criar uniões de tipos ou tipos de funções.

```
interface Usuario {  
  id: number;  
  nome: string;  
  email?: string; // Propriedade opcional  
  readonly token: string; // Apenas leitura  
}  
  
type Resposta = Usuario | string; // Union Type
```


| FLEXIBILIDADE COM GENERICS

Generics permitem criar componentes reutilizáveis que funcionam com qualquer tipo, mantendo a integridade. O tipo T funciona como uma variável para o tipo real.

```
// Interface genérica para qualquer resposta de API  
interface ApiResponse<T> {  
    dados: T;  
    sucesso: boolean;  
}  
  
// Reuso dinâmico  
type UserRes = ApiResponse<Usuario>;  
type ProdRes = ApiResponse<Produto[]>;
```


| ORIENTAÇÃO A OBJETOS NO TS

O TypeScript expande as classes com modificadores de acesso: `public`, `private` (acesso interno) e `protected` (acesso em subclasses).

```
class Loja {  
    // Shorthand: declara e inicializa no construtor  
    constructor(  
        public nome: string,  
        private faturamento: number  
    ) {}  
  
    obterRelatorio(): string {  
        return `Loja: ${this.nome}`;  
    }  
}
```


| TIPOS UTILITÁRIOS NATIVOS

Transformam tipos existentes para evitar duplicação. Comuns em atualizações parciais de dados ou listagens resumidas.

```
interface Tarefa { id: number; texto: string; ok: boolean; }
```

```
// Partial: Torna tudo opcional (útil para Updates)
```

```
type UpdateTarefa = Partial<Tarefa>;
```

```
// Omit: Remove propriedades específicas
```

```
type NovaTarefa = Omit<Tarefa, "id">;
```

```
// Pick: Seleciona apenas o necessário
```

```
type Resumo = Pick<Tarefa, "texto">;
```


| TYPESCRIPT NO DOM

Como o TS não sabe o que há no HTML, usamos **Type Assertion** (as) para informar o tipo específico do elemento e garantir acesso às suas propriedades.

```
// Informamos que o elemento é um Input para acessar '.value'  
const inputCep = document.getElementById("cep") as HTMLInputElement;  
  
if (inputCep) {  
  console.log(inputCep.value);  
}  
  
// Eventos com tipos explícitos  
inputCep.addEventListener("blur", (e: FocusEvent) => {  
  const el = e.target as HTMLInputElement;  
  console.log(el.value);  
});
```




Segurança em Primeiro Lugar

A migração para o TypeScript é um investimento na longevidade do projeto e na produtividade do desenvolvedor.